

Atelier 06 : Spring MVC Coté Serveur :

Formulaires et Validation

Cet Atelier concerne la création des formulaires pour ajouter et modifier des produits ainsi que la validation des données.

Objectifs :

1. Créer un formulaire pour ajouter des produits,
2. La validation des données,
3. Créer un formulaire pour modifier les produits,
4. Utiliser le même formulaire pour la création et la modification,
5. Affecter une catégorie à un produit.

Créer un formulaire pour ajouter des produits

1. Créer, dans le dossier `src/main/resources/templates` le fichier `createProduit.html` :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout" >
<link rel="stylesheet"
type="text/css"href="webjars/bootstrap/4.3.1/css/bootstrap.min.css" />
<head>
<meta charset="utf-8">
<title>Ajout des Produits</title>
</head>
<body>
  <div th:replace="template"></div>
  <div class="container mt-5">
    <div class="card">
      <div class="card-header"> Ajout des Produits </div>
      <div class="card-body">
        <form th:action="@{saveProduit}" method="post">
          <div class="form-group">
            <label class="control-label">Nom Produit :</label>
            <input type="text" name="nomProduit" class="form-control" />
          </div>
          <div class="form-group">
            <label class="control-label">Prix Produit :</label>
            <input type="text" name="prixProduit" class="form-control" />
          </div>
          <div class="form-group">
            <label class="control-label">date création :</label>
            <input type="date" name="dateCreation" class="form-control" />
          </div>
        </form>
      </div>
    </div>
    <button type="submit" class="btn btn-primary">Valider</button>
  </div>
</body>
```

```
</html>
```

2. Modifier la méthode `saveProduit` de la classe `ProduitController` comme suit :

```
@RequestMapping("/saveProduit")
public String saveProduit(@ModelAttribute("produit") Produit produit)
{
    produitService.saveProduit(produit);
    return "createProduit";
}
```

3. Ajouter les annotations à l'attribut `dateCreation` de l'entité `Produit` :

```
@Temporal(TemporalType.DATE)
@DateTimeFormat(pattern = "yyyy-MM-dd")
private Date dateCreation;
```

La validation des données

4. Ajouter la dépendance `spring-boot-starter-validation`

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

5. Editer l'entité `Produit` et ajouter les contraintes de validation suivantes :

```
import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
...

@NotNull
@Size (min = 4,max = 15)
private String nomProduit;

@Min(value = 10)
@Max(value = 10000)
private Double prixProduit;

@Temporal(TemporalType.DATE)
@DateTimeFormat(pattern = "yyyy-MM-dd")
@PastOrPresent
private Date dateCreation;
```

6. Modifier les méthodes `showCreate` et `saveProduit` de la classe `ProduitController` :

```
@RequestMapping("/showCreate")
public String showCreate(ModelMap modelMap)
{
    modelMap.addAttribute("produit", new Produit());
    return "createProduit";
}
```

```

@RequestMapping("/saveProduit")
public String saveProduit(@Valid Produit produit,
                          BindingResult bindingResult)
{
    if (bindingResult.hasErrors()) return "createProduit";

    produitService.saveProduit(produit);
    return "createProduit";
}

```

7. Modifier le fichier createProduit.html :

```

<form th:action="@{saveProduit}" method="post">
    <div class="form-group">
        <label class="control-label">Nom Produit :</label>
        <input type="text" name="nomProduit" class="form-control" />
        <span th:errors=${produit.nomProduit} class="text-danger"> </span>
    </div>
    <div class="form-group">
        <label class="control-label">Prix Produit :</label>
        <input type="text" name="prixProduit" class="form-control" />
        <span th:errors=${produit.prixProduit} class="text-danger"> </span>
    </div>
    <div class="form-group">
        <label class="control-label">date création :</label>
        <input type="date" name="dateCreation" class="form-control" />
        <span th:errors=${produit.dateCreation} class="text-danger"> </span>
    </div>
    <div>
        <button type="submit" class="btn btn-primary">Ajouter</button>
    </div>
</form>

```

8. Tester la validation des données

The screenshot shows a web browser window with the address bar displaying 'localhost:8080/produits/saveProduit'. The page has a dark header with the name 'Nadhem BelHadj' and a 'Produits' dropdown menu. The main content area is titled 'Ajout des Produits' and contains a form with three input fields and an 'Ajouter' button. The first field, 'Nom Produit', contains 'abc' and has a red error message below it: 'la taille doit être comprise entre 4 et 25'. The second field, 'Prix Produit', contains '5.0' and has a red error message below it: 'doit être supérieur à ou égal à 10.0'. The third field, 'date création', contains 'jj/mm/aaaa' and has a red error message below it: 'doit être une date dans le passé ou le présent'.

Créer un formulaire pour modifier les produits

9. Modifier la page listeProduit.html pour ajouter le lien « Editer » :

```
<td><a class="btn btn-success"
th:href="@{modifierProduit(id=${p.idProduit})}">Editer</a></td>
```

10. Créer la page editProduit.html :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">
<link rel="stylesheet" type="text/css"
href="/webjars/bootstrap/4.3.1/css/bootstrap.min.css" />
<head>
<meta charset="utf-8">
<title>Ajout des Produits</title>
</head>
<body>
<div th:replace="template"></div>
<div class="container mt-5">
<div class="card">
  <div class="card-header">
    Modifier des Produits
  </div>
  <div class="card-body">
<form th:action="@{saveProduit}" method="post">
  <div class="form-group">
    <label class="control-label" hidden>ID Produit :</label>
    <input type="hidden" name="idProduit" class="form-control"
th:value="${produit.idProduit}" />
  </div>
  <div class="form-group">
    <label class="control-label">Nom Produit :</label>
    <input type="text" name="nomProduit" class="form-control"
th:value="${produit.nomProduit}" />
    <span th:errors=${produit.nomProduit} class="text-danger"> </span>
  </div>
  <div class="form-group">
    <label class="control-label">Prix Produit :</label>
    <input type="text" name="prixProduit" class="form-control"
th:value="${produit.prixProduit}" />
    <span th:errors=${produit.prixProduit} class="text-danger"> </span>
  </div>
  <div class="form-group">
    <label class="control-label">date création :</label>
    <input type="date" name="dateCreation" class="form-control"
th:value="${produit.dateCreation}" />
    <span th:errors=${produit.dateCreation} class="text-danger"> </span>
  </div>
  <div>
<button type="submit" class="btn btn-primary">Valider</button>
  </div>
</form>
</div>
</div>
</body>
</html>
```

Utiliser le même formulaire pour la création et la modification

1. Renommer le fichier createProduit.html en formProduit.html
2. Au niveau de la classe ProduitController, modifier les méthodes showCreate et saveProduit :

```
@RequestMapping("/showCreate")
public String showCreate(ModelMap modelMap)
{
    modelMap.addAttribute("produit", new Produit());
    return "formProduit";
}

@RequestMapping("/saveProduit")
public String saveProduit(@Valid Produit produit,
                          BindingResult bindingResult)
{
    if (bindingResult.hasErrors()) return "formProduit";

    produitService.saveProduit(produit);
    return "formProduit";
}
```

3. Tester l'ajout d'un produit,
4. Au niveau de la classe ProduitController, modifier editProduit :

```
@RequestMapping("/modifierProduit")
public String editProduit(@RequestParam("id") Long id, ModelMap modelMap)
{
    Produit p = produitService.getProduit(id);
    modelMap.addAttribute("produit", p);
    modelMap.addAttribute("mode", "edit");
    return "formProduit";
}

@RequestMapping("/showCreate")
public String showCreate(ModelMap modelMap)
{
    modelMap.addAttribute("produit", new Produit());
    modelMap.addAttribute("mode", "new");
    return "formProduit";
}
```

5. Modifier le fichier formProduit.html comme suit :

```
...
<div class="card">
    <div class="card-header" th:if="${mode=='new'}"> Ajout des Produits </div>
<div class="card-header" th:if="${mode=='edit'}">Modification des Produits </div>
    <div class="card-body">
        <form th:action="@{saveProduit}" method="post">
            <div class="form-group">
                <label class="control-label" hidden>ID Produit :</label>
                <input type="hidden" name="idProduit" class="form-control"
                    th:value="${produit.idProduit}" />
            </div>
        </form>
    </div>
...

```

Affecter une catégorie à un produit

11. Vérifiez que vous avez bien créé l'entité « Catégorie » dans l'atelier
« Persister et interroger les données avec Spring Data JPA »

```
package com.nadhem.produits.entities;

import java.util.List;
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor

@Entity
public class Catégorie {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idCat;
    private String nomCat;
    private String descriptionCat;

    @JsonIgnore
    @OneToMany(mappedBy = "categorie")
    private List<Produit> produits;
}
```

L'entité Produit

```
@Entity
public class Produit {
    ...

    @ManyToOne
    private Catégorie categorie;

    ...
}
```

Créer l'interface `CategorieRepository`

```
package com.nadhem.produits.repos;

import org.springframework.data.jpa.repository.JpaRepository;
import com.nadhem.produits.entities.Categorie;

public interface CategorieRepository extends JpaRepository<Categorie, Long>
{
}
```

12. Ajouter au service la méthode `getAllCategories`:

Interface `ProduitService`

```
List<Categorie> getAllCategories();
```

Classe `ProduitServiceImpl`

```
@Autowired
CategorieRepository categorieRepository;

@Override
public List<Categorie> getAllCategories() {
    return categorieRepository.findAll();
}
```

13. Modifier la vue `formProduit.html` comme suit :

```
<div class="form-group">
  <label class="control-label">Categorie </label>
  <select name="categorie" class="form-control"
    th:value="${produit.categorie.idCat}"
    th:if="${!#strings.isEmpty(produit.categorie)}" >
    <option th:each="c:${categories}" th:if="${mode=='edit'}"
      th:value="${c.idCat}" th:text="${c.nomCat}"
      th:selected="${ c.idCat==produit.categorie.idCat}">
    </option>
  </select>
  <!-- s'il s'agit d'un ajout (mode==new) ou d'une modification d'un produit ayant une catégorie null -->
  <select name="categorie" class="form-control"
    th:if="${#strings.isEmpty(produit.categorie)}">
    <option th:each="c:${categories}" th:value="${c.idCat}"
      th:text="${c.nomCat}">
    </option>
  </select>
</div>
```

14. Ajouter à la vue listeProduits.html :

```
<td th:if= "${!#strings.isEmpty(p.categorie)}" th:text="${p.categorie.nomCat}"></td>
<td th:if= "${#strings.isEmpty(p.categorie)}" th:text="${'Pas de Catégorie'}"></td>
```

15. Modifier le Controlleur :

```
@RequestMapping("/showCreate")
public String showCreate(ModelMap modelMap)
{
    List<Categorie> cats = produitService.getAllCategories();
    modelMap.addAttribute("produit", new Produit());
    modelMap.addAttribute("mode", "new");
    modelMap.addAttribute("categories", cats);
    return "formProduit";
}

@RequestMapping("/modifierProduit")
public String editProduit(@RequestParam("id") Long id, ModelMap modelMap)
{
    Produit p = produitService.getProduit(id);
    List<Categorie> cats = produitService.getAllCategories();
    modelMap.addAttribute("produit", p);
    modelMap.addAttribute("mode", "edit");
    modelMap.addAttribute("categories", cats);

    return "formProduit";
}
```

Pour revenir à la liste des produits à la suite d'un ajout ou modification, utilisez « redirect »

```
@RequestMapping("/saveProduit")
public String saveProduit(@Valid Produit produit, BindingResult bindingResult)
{
    if (bindingResult.hasErrors()) return "formProduit";

    produitService.saveProduit(produit);
    //return "ListeProduits";
    return ("redirect:/ListeProduits");
}
```


Corriger la pagination

ListeProduits.html

```
<td><a class="btn btn-success"
th:href="@{modifierProduit(id=${p.idProduit},page=${currentPage},size=${size})}"
>Editer</a></td>
```

Formproduit.html

```
<input name="page" class="form-control" th:value="${page}" />
<input name="size" class="form-control" th:value="${size}" />
```

ProduitController

```
@RequestMapping("/saveProduit")
public String saveProduit(@Valid Produit produit, BindingResult bindingResult,
                          @RequestParam (name="page",defaultValue = "0") int page,
                          @RequestParam (name="size",defaultValue = "2") int size)
{
    int currentPage;
    boolean isNew = false;
    if (bindingResult.hasErrors()) return "formProduit";

    if (produit.getIdProduit()==null) //ajout
        isNew=true;

    produitService.saveProduit(produit);
    if (isNew) //ajout
    {
        Page<Produit> prods = produitService.getAllProduitsParPage(page, size);
        currentPage = prods.getTotalPages()-1;
    }
    else //modif
        currentPage=page;

    return ("redirect:/ListeProduits?page="+currentPage+"&size="+size);
}
```